Árvores balanceadas

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

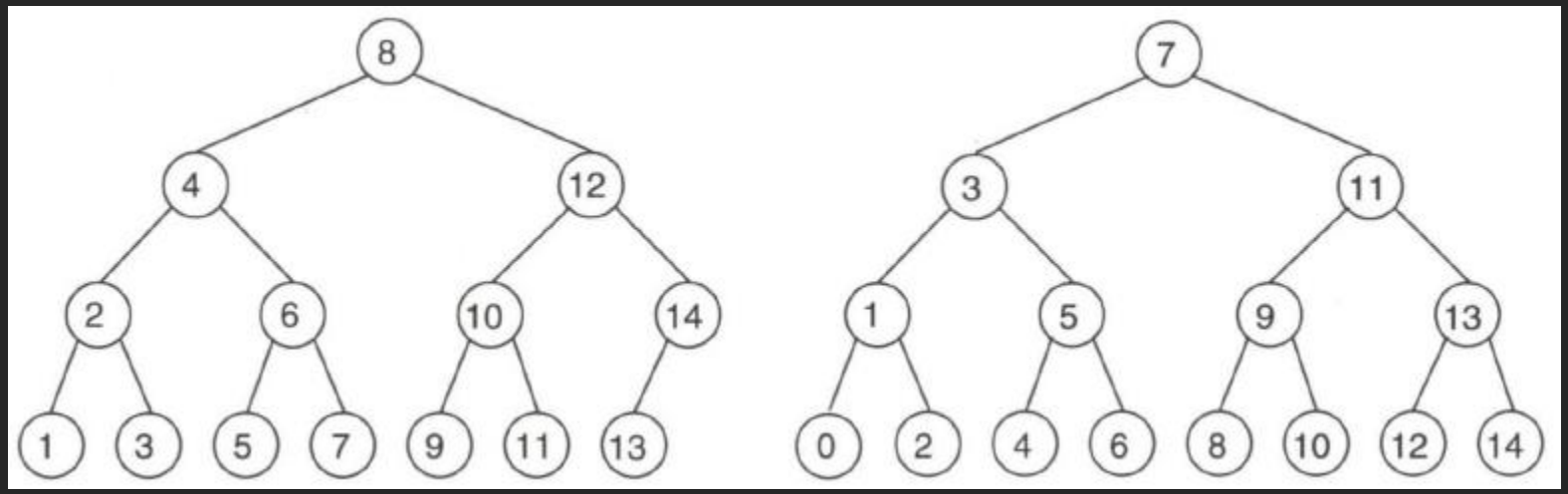
- Introdução
- Conceito de balanceamento
- Árvores AVL
- Árvores B
- Exercícios

Introdução

- Árvore binária de busca: minimizar custo da busca
  - Árvore criada estaticamente: pode ter altura mínima
    - Custo da busca: O(lg n)
  - Após sequência de inserções/remoções: pode degenerar-se em ziguezague (lista)
    - Custo da busca: O(n)
- Árvore balanceada: matém altura sempre O(lg n) após inserções/remoções
  - Custo da busca: O(lg n)
  - Custo da inserção/remoção inclui custo de manutenção da altura: O(lg n)

Conceito de balanceamento

- Árvore completa possui altura mínima: $1 + \lfloor \lg n \rfloor$
  - Manter árvore completa: requer $\Omega(n)$ em algumas situações
    - Ex.: Inserção da chave 0 na árvore completa a seguir
      - Para torná-la completa novamente: mudança em todos os nós



Conceito de balanceamento

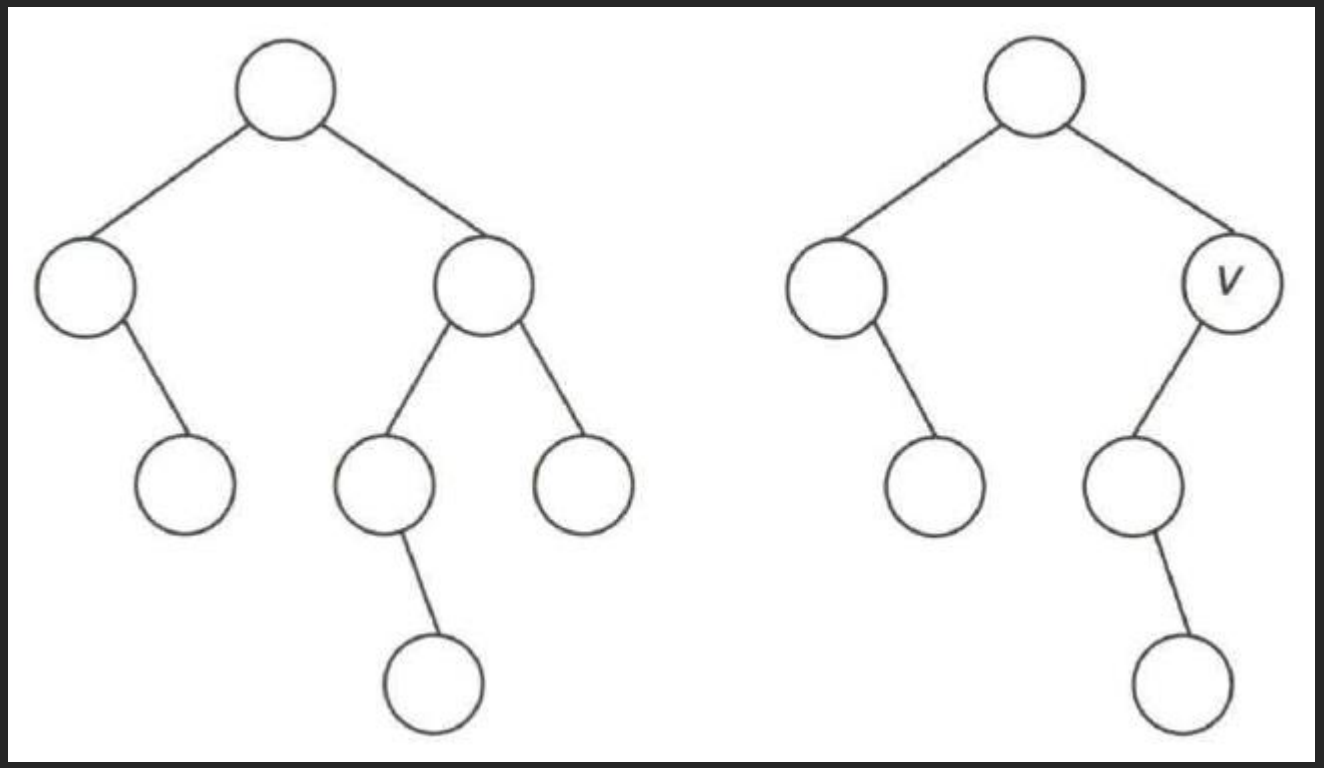
- Custo $\Omega(n)$ para manter árvore: muito alto
  - Inserções e remoções ficariam mais caras
  - Árvores completas são, portanto, inviáveis
- Alternativa: não exigir árvore de altura mínima
  - Mas manter ordem de grandeza: O(lg n)
- Árvore balanceada: possui altura h = O(lg n)
  - Desejável: subárvores com m nós possuem altura O(lg m)
  - Mais fáceis de manter que uma árvore completa
  - Exemplos: AVL, graduada, rubro-negra, B

Árvores AVL

- *"Uma árvore binária T é AVL quando, para qualquer nó de T, as alturas de suas duas subárvores, esquerda e direita, diferem em módulo de até uma unidade"*
  - Nó regulado: satisfaz essa condição
  - Nó desregulado: não satisfaz
- Uma árvore completa é AVL
  - Recíproca nem sempre é verdadeira
- Árvores AVL são balanceadas

Árvores AVL

- Exemplos de árvores AVL e não AVL:



Árvores AVL

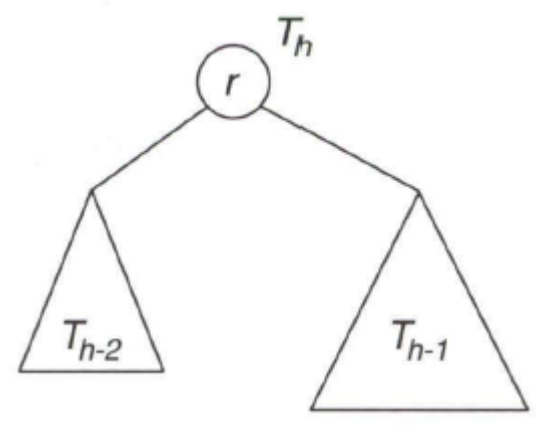
- Demonstração de que árvores AVL são balanceadas:
  - Estratégia 1: dada árvore AVL com n nós, qual a altura h máxima?
    - Verificar se h = O(lg n)
  - Estratégia 2: dada árvore AVL com altura h, qual o menor número de nós n possível?
    - Verificar se h = O(lg n)
    - Estratégia adotada

Árvores AVL

- (cont.)
  - Seja T_h uma árvore AVL com altura h e menor número de nós possível:
    - Se h = 0: árvore vazia
    - Se h = 1: só existe a raiz
    - Se h > 1: raiz tem duas subárvores T_{h-1} e T_{h-2}
      - Se as duas subárvores tivessem altura e T_{h-1} então n não seria mínimo nessa árvore
      - Se alguma subárvore tivesse algura menor que h-2, então T_h não seria AVL
    - T_{h-1} e T_{h-2} podm ser construídas de forma recursiva...

Árvores AVL

- (cont.)
  - Ilustração de T_h com subárvores T_{h-1} e T_{h-2}



Árvores AVL

- (cont.)
  - Seja $|T_h|$ o número de nós da árvore T_h:

$$\begin{cases} |T_h| = 0, & \text{para} \quad h = 0 \\ |T_h| = 1, & h = 1 \\ |T_h| = 1 + |T_{h-1}| + |T_{h-2}|, & h > 1 \end{cases}$$

  - Note que o h-ésimo número de Fibonacci F_h é dado por:

$$\begin{cases} F_h = 0, & \text{para} \quad h = 0 \\ F_h = 1, & h = 1 \\ F_h = F_{h-1} + F_{h-2}, & h > 1 \end{cases}$$

  - Quando h > 1:

$$F_h = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^h - \left(\frac{1-\sqrt{5}}{2} \right)^h \right]$$

Árvores AVL

- (cont.)
  - Observando que $|T_h| \geq F_h$:

Como $h > 0$, $\frac{1}{\sqrt{5}}(\frac{1-\sqrt{5}}{2})^h < 1$. Portanto, $|T_h| > \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^h - 1$.

Fazendo $a = (1+\sqrt{5})/2$ tem-se $|T_h| > \frac{1}{\sqrt{5}}\, a^h - 1$.

Ou seja, $\log_a(|T_h| + 1) > h - \log_a \sqrt{5}$

$h < \log_a(|T_h| + 1) + \log_a \sqrt{5} = O(\log n)$.

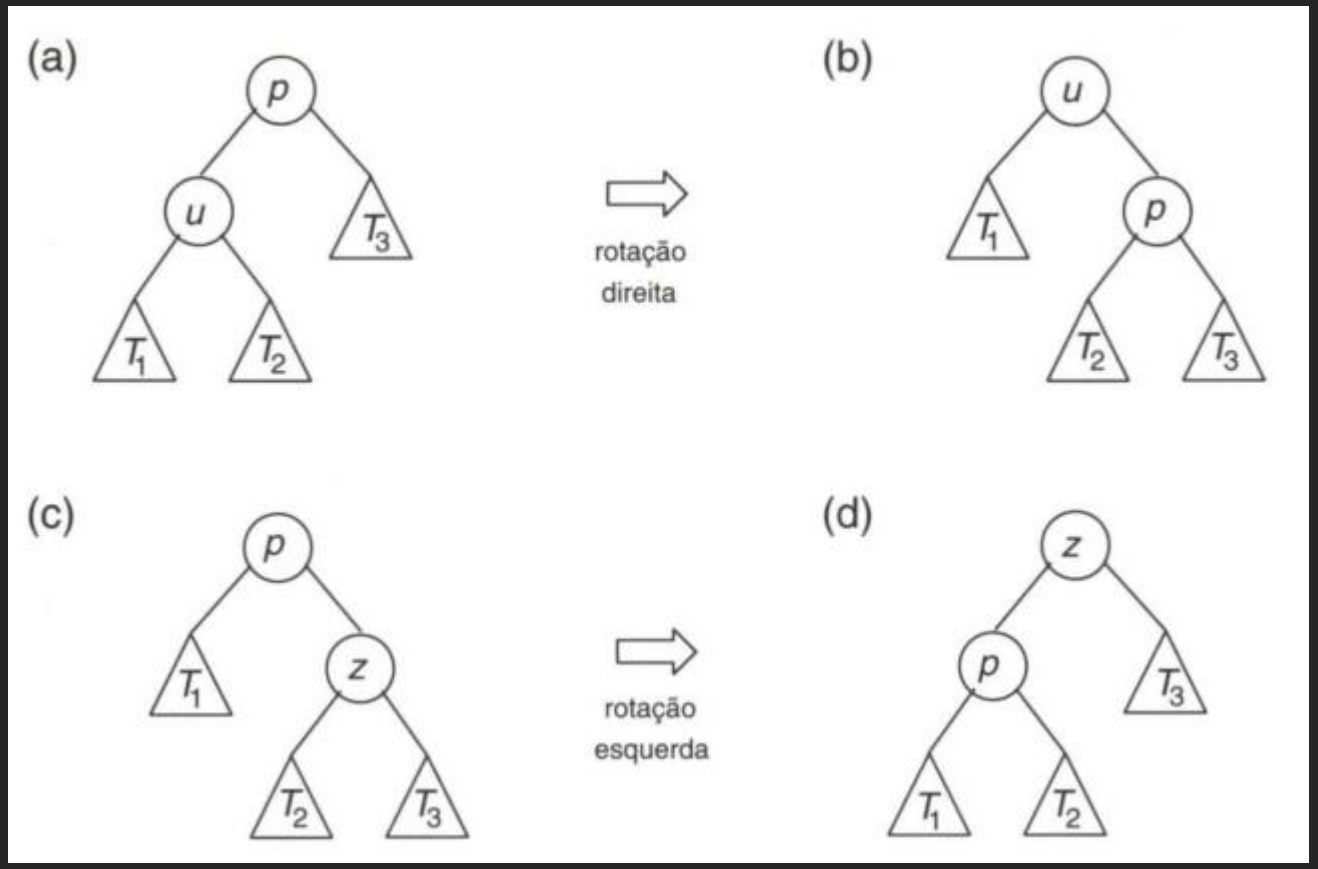
Esse resultado indica que a árvore AVL é, de fato, balanceada.

Árvores AVL

- Inserção em árvore AVL pode deixar nó desregulado
  - Nesse caso: aplicar transformações na árvore
- Quatro transformações: rotação direita, rotação esquerda, rotação dupla direita e rotação dupla esquerda
  - Nó raiz da transformação: nó onde ela é aplicada
  - Árvore continua sendo binária de busca

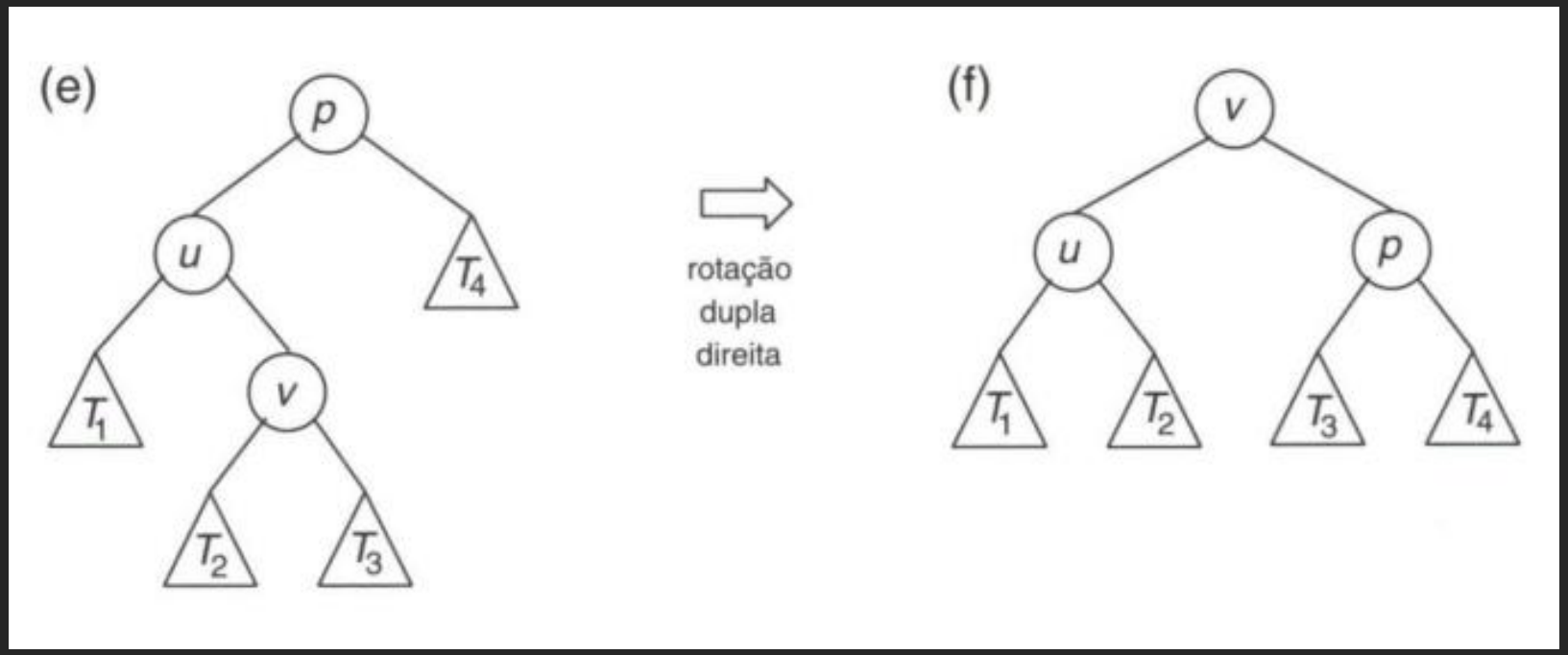
Árvores AVL

- Rotação direita e esquerda no nó p:



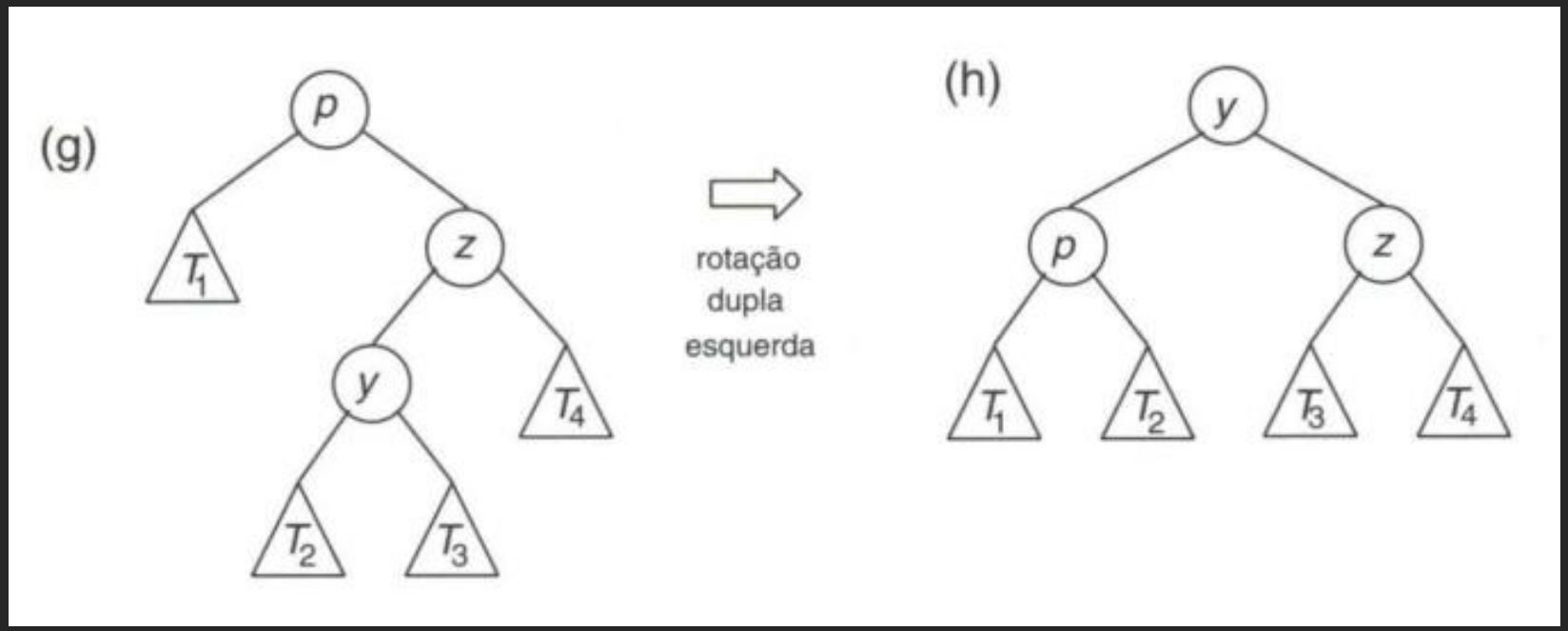
Árvores AVL

- Rotação dupla direita no nó p:
  - Pode ser obtida através de duas rotações simples: rotação esquerda em u seguida de rotação direita em p



Árvores AVL

- Rotação dupla esquerda no nó p:
  - Pode ser obtida através de duas rotações simples: rotação direita em z seguida de rotação esquerda em p



Árvores AVL

- Considere a inserção de um nó q em uma árvore AVL:
  - Se todos os nós continuam regulados: árvore continua AVL
  - Caso contrário, considere p o nó mais próximo às folhas que se tornou desregulado
    - p está no caminho de q até a raiz
    - $|h_E(p) - h_D(p)| = 2$, pois antes a árvore era AVL, agora não é mais e a inserção pode acrescentar, no máximo, 1 unidade na altura
    - Há quatro situações a se analisar:

Árvores AVL

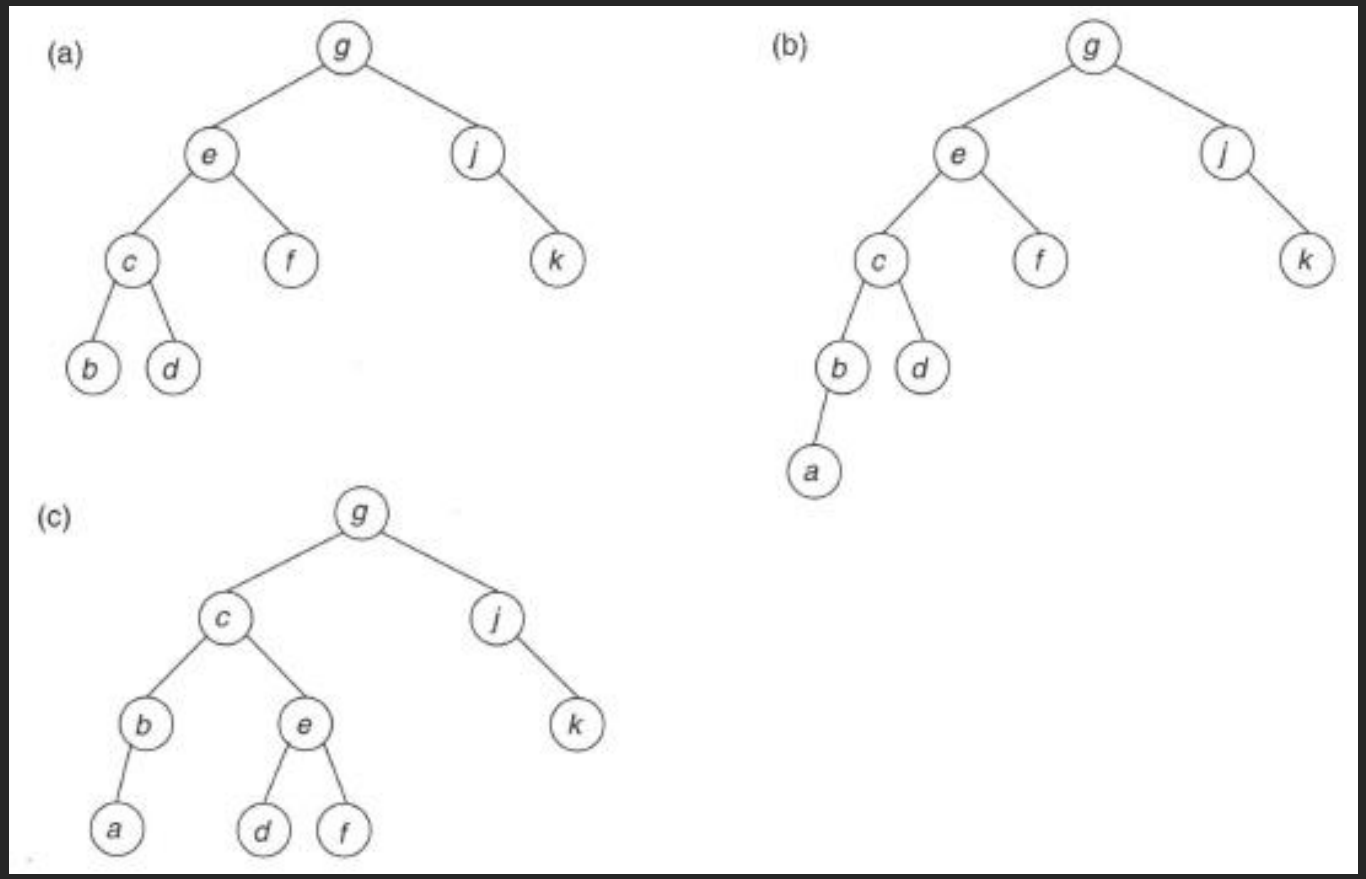
- Caso 1: $h_E(p) > h_D(p)$
  - Então q pertence à subárvore esquerda de p, pois essa é a subárvore de maior altura que deixou p desregulado
  - p também possui filho esquerdo u ≠ q, pois caso contrário p estaria regulado (diferença de no máximo 1 na altura)
  - Há duas possibilidades a considerar:
    - Caso 1.1: $h_E(u) > h_D(u)$
      - Essa situação corresponde à parte (a) da figura e uma rotação direita deixa p regulado
    - Caso 1.2: $h_D(u) > h_E(u)$
      - Essa situação corresponde à parte (e) da figura e uma rotação dupla direita deixa p regulado

Árvores AVL

- Caso 2: $h_D(p) > h_E(p)$
  - Então q pertence à subárvore direita de p, pois essa é a subárvore de maior altura que deixou p desregulado
  - p também possui filho direito $z \neq q$, pois caso contrário p estaria regulado (diferença de no máximo 1 na altura)
  - Há duas possibilidades a considerar:
    - Caso 2.1: $h_D(z) > h_E(z)$
      - Essa situação corresponde à parte (c) da figura e uma rotação esquerda deixa p regulado
    - Caso 2.2: $h_E(z) > h_D(z)$
      - Essa situação corresponde à parte (g) da figura e uma rotação dupla esquerda deixa p regulado
- Após transformações, a altura da subárvore com raiz em p, que tinha aumentado em 1 unidade com a inserção, voltou a diminuir 1 unidade: ancestrais de p continuam regulados

Árvores AVL

- Exemplo: inserção do nó ***a*** seguida de regulagem



Árvores AVL

- Implementação da inserção:
  - Novo campo em cada nó da árvore: balanço
  - Para um dado nó v: balanço(v) = $h_D(v) - h_E(v)$
    - v está regulado se $-1 \leq$ balanço(v) ≤ 1
  - Atualização do balanço(v) durante inserção
    - Se a inserção de q aumenta altura da subárvore esquerda com raiz em v: balanço(v) é decrementado
      - Se balanço(v) = -2: v ficou desregulado
    - Se inserção aumenta altura da subárvore direita com raiz em v: balanço(v) é incrementado
      - Se balanço(v) = 2: v ficou desregulado

Árvores AVL

- Após a inserção de q: nó w, pai de q, teve altura da subárvore esquerda ou direita alterada
  - balanço(w) foi alterado
  - Essa alteração pode afetar outros nós v de w até a raiz da árvore
    - Deve ser feita análise recursiva para todos os nós ancestrais
      - Início em w
      - Continua em direção à raiz, até que encontre nó v onde T_v não teve altura alterada

Árvores AVL

- Supondo inserção de q na subárvore esquerda de v, há três casos a considerar:
  - Caso 1: balanço(v) = 1 antes da inserção
    - balanço(v) = 0, altura de T_v não foi alterada
    - Nesse caso, ancestrais de v não são analisados
  - Caso 2: balanço(v) = 0 antes da inserção
    - balanço(v) = -1, altura de T_v foi alterada
    - Nesse caso, nós ancestrais de v devem ser analisados

Árvores AVL

- Caso 3: balanço(v) = -1 antes da inserção
  - balanço(v) = -2, e nó v está desregulado
  - Nesse caso:
    - Deve ser feita a rotação adequada (altura após inserção = altura antes da inserção)
    - Alturas dos ancestras de v não se alteram e ancestrais não são analisados
- Supondo inserção de q na subárvore direita de v, há três casos simétricos a considerar

Árvores AVL

- Algoritmo recursivo ins-AVL(x, pt, h)
  - Combina busca e inserção no mesmo procedimento
    - Busca-se pela chave x
    - Durante a busca é conhecido o caminho do novo nó pt até a raiz
    - Se x não existe, novo nó é alocado
    - Após inserção, caminho inverso é feito (volta da recursão)
      - balanço é acertado: campo bal
      - Se necessária, rotação adequada é realizada
      - Parâmetro lógico h indica se nós ancestrais devem ser analisados
  - Chamada externa: ins-AVL(x, ptraiz, h)
    - ptraiz aponta para a raiz da árvore

Árvores AVL

```
procedimento ins-AVL(x, pt, h)
    se pt = λ então
        início-no(pt)
        h := V
    senão se x = pt ↑ .chave então pare
        se x < pt ↑ .chave então
            ins-AVL(x, pt ↑ .esq, h)
            se h então
                caso pt ↑ .bal seja
                 1:    pt ↑ .bal := 0;    h := F
                 0:    pt ↑ .bal := −1
                -1:    caso1(pt, h)          % rebalanceamento
        senão ins-AVL(x, pt ↑ .dir, h)
            se h então
                caso pt ↑ .bal seja
                -1:    pt ↑ .bal := 0;    h := F
                 0:    pt ↑ .bal := 1
                 1:    caso2(pt, h)          % rebalanceamento
```

Árvores AVL

- Procedimento auxiliar inicio-no(pt):

```
procedimento inicio-no(pt)
    ocupar(pt)
    pt ↑ .esq := λ;    pt ↑ .dir := λ
    pt ↑ .chave := x;    pt ↑ .bal := 0
```

Árvores AVL

- Procedimento auxiliar caso1(pt, h): realiza rotação direita (caso 1.1) ou rotação dupla direita (caso 1.2)

```
procedimento caso1(pt, h)
    ptu := pt ↑ .esq
    se ptu ↑ .bal = −1 então
            pt ↑ .esq := ptu ↑ .dir;    ptu ↑ .dir := pt
            pt ↑ .bal := 0;    pt := ptu
    senão ptv := ptu ↑ .dir
            ptu ↑ .dir := ptv ↑ .esq;    ptv ↑ .esq := ptu
            pt ↑ .esq := ptv ↑ .dir;    ptv ↑ .dir := pt
            se ptv ↑ .bal = −1 então pt ↑ .bal := 1 senão pt ↑ .bal := 0
            se ptv ↑ .bal = 1 então ptu ↑ .bal := −1 senão ptu ↑ .bal := 0
            pt := ptv
    pt ↑ .bal := 0;    h := F
```

Árvores AVL

- Procedimento auxiliar caso2(pt, h): realiza rotação esquerda (caso 2.1) ou rotação dupla esquerda (caso 2.2)

```
procedimento caso2(pt, h)
    ptu := pt ↑ .dir
    se ptu ↑ .bal = 1 então
            pt ↑ .dir := ptu ↑ .esq;    ptu ↑ .esq := pt
            pt ↑ .bal := 0;    pt := ptu
    senão ptv := ptu ↑ .esq
            ptu ↑ .esq := ptv ↑ .dir;    ptv ↑ .dir := ptu
            pt ↑ .dir := ptv ↑ .esq;    ptv ↑ .esq := pt
            se ptv ↑ .bal = 1 então pt ↑ .bal := −1 senão pt ↑ .bal := 0
            se ptv ↑ .bal = −1 então ptu ↑ .bal := 1 senão ptu ↑ .bal := 0
            pt := ptv
    pt ↑ .bal := 0;    h := F
```

Árvores AVL

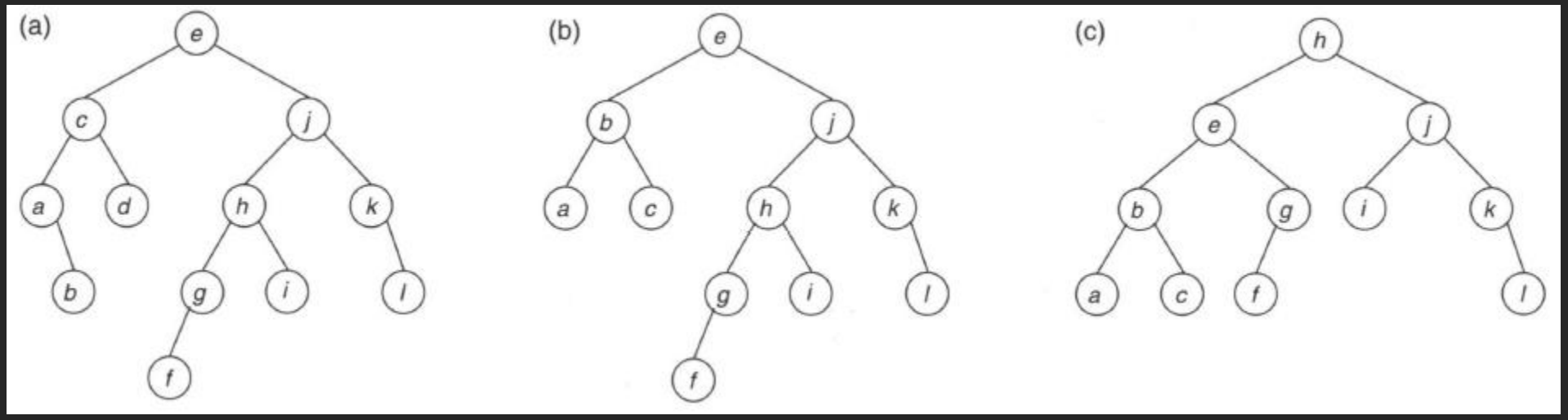
- Complexidade da inserção: O(lg n)
  - Altura de uma árvore AVL: O(lg n) por ser balanceada
  - Busca: O(lg n)
  - Correção do balanço: volta da recursão
    - Cada correção: O(1)
    - Corrige no máximo O(lg n) nós: caminho até a raiz
  - Uma rotação, se necessária: O(1)

Árvores AVL

- Remoção:
  - Aplicar a remoção de árvore binária de busca
  - Verificar se a árvore ficou desregulada:
    - Analisar nós no caminho até a raiz
    - Podem ser utilizadas até O(lg n) rotações!

Árvores AVL

- Exemplo de remoção:
  - Nó d é removido
  - Nó c fica desregulado: rotação dupla direita
  - Nó e fica desregulado: rotação dupla esquerda



Árvores B

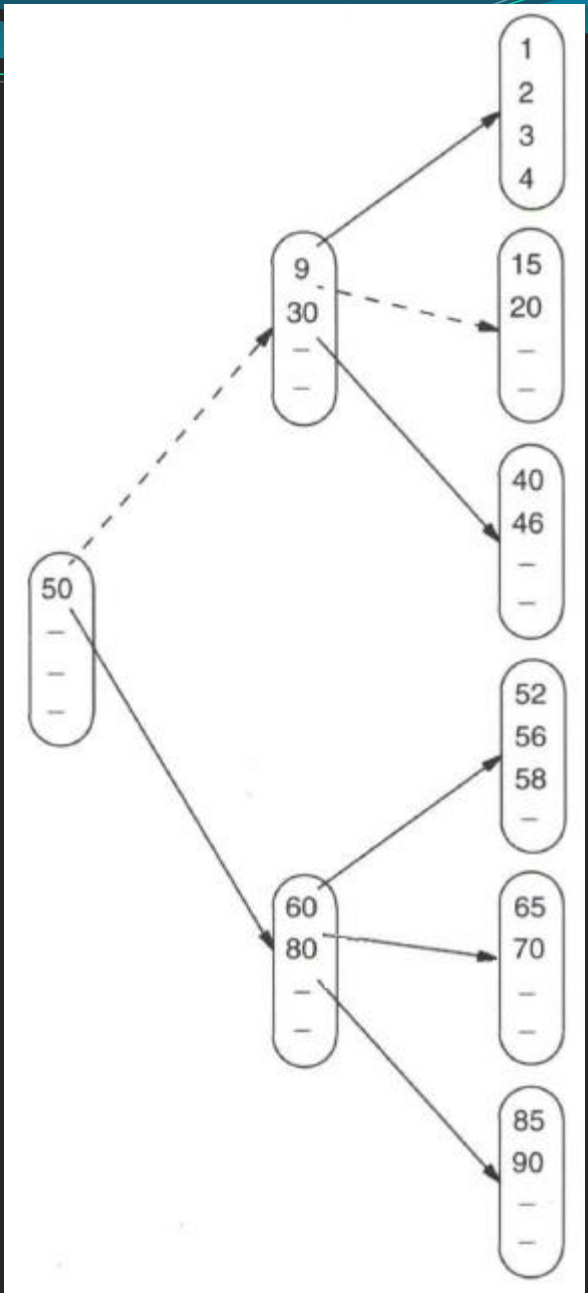
- Para muitas aplicações, quantidade de nós é enorme
  - Memória principal não permite armazenar tudo
  - Solução: manter árvore na memória secundária (disco)
    - Acesso a um nó: muito demorado
    - Necessidade: minimizar tempo de acesso (busca/inserção/remoção)
- Árvore B: mantém mais de uma chave em cada nó
  - Adequadas para lidar com memória secundária: empregadas em BD
  - Operações de busca/inserção/remoção rápidas

Árvores B

- Uma árvore B de ordem d, onde d é número natural, satisfaz as seguintes condições:
  i. A raiz é uma folha ou tem pelo menos dois filhos;
  ii. Cada nó, exceto raiz e folhas, possui no mínimo d+1 filhos
  iii. Cada nó tem no máximo 2d+1 filhos;
  iv. Todas as folhas estão no mesmo nível
- Nó de uma árvore B é chamado de página

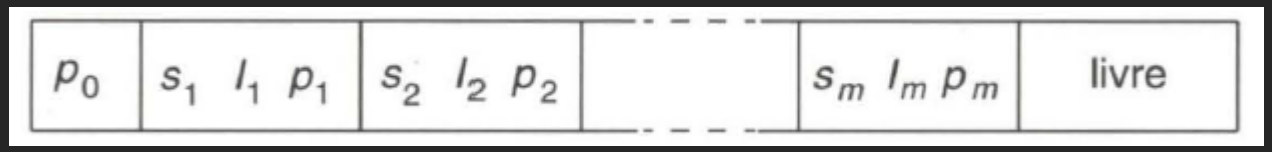
Árvores B

- Exemplo: árvore B de grau 2



Árvores B

- Em uma página P de uma árvore B de ordem d:
  - Se P tem m chaves, então possui m+1 filhos
    - P possui m+1 ponteiros $p_0, p_1, \ldots, p_m$ para os filhos de P
  - Raiz possui entre 1 e 2d chaves
  - Demais páginas possuem entre d e 2d chaves
  - As chaves $s_1, s_2, \ldots, s_m$ estão ordenadas dentro da página
  - I_k é a informação associada à chave s_k
  - A tripla (s_k, I_k, p_k) ou o par (s_k, p_k) é chamado de entrada



Árvores B

- Em uma página P com m chaves:
  - Se a chave y pertence à página apontada por p_0 então $y < s_1$
  - Se y pertence à página apontada por p_k, $1 \leq k \leq m-1$ então $s_k < y < s_{k+1}$
  - Se a chave y pertence à página apontada por p_m então $y > s_m$
- Altura de uma árvore B: $\Theta(\lg n)$
  - $\log_{2d+1}(n+1) \leq h \leq 1 + \log_{d+1}\left(\frac{n+1}{2}\right)$

Árvores B

- Busca pela chave x: caminho da raiz em direção às folhas
  - Verificam-se as chaves na página atual
  - Se x não foi encontrada, busca prossegue na página adequada do próximo nível
  - Complexidade: O(lg n)

Árvores B

- Algoritmo BuscaB:
  - Dado um ponteiro p para página com m chaves
    - Chaves da página p: p↑.s[1], ..., p↑.s[m]
  - Ponteiros para as páginas filhas: p↑.pont[0], ..., p↑.pont[m]
  - Parâmetros pt, f e g
    - Se f = 1, chave foi encontrada em pt ↑.s[g]
    - Se f = 0, pt aponta para a última página analisada (folha) e g possui posição onde x seria incluída

Árvores B

```
algoritmo 5.3: Busca da chave x numa árvore B
procedimento buscaB(x, pt, f, g)
    p := ptraiz;   pt := λ;   f := 0
    enquanto p ≠ λ faça
        i := g := 1;   pt := p
        enquanto i ≤ m faça
            se x > p↑.s[i] então i := g := i + 1
            senão se x = p↑.s[i] então
                        p := λ                % chave encontrada
                        f := 1
                    senão p := p↑.pont[i − 1]% mudança de página
                    i := m + 2
        se i = m + 1 então p := p↑.pont[m]
```

Árvores B

- Inserção da chave x:
  - Realiza busca por x
  - Inclusão deverá ocorrer na g-ésima posição folha apontada por pt
  - Se folha já possui 2d chaves (cheia) é necessário fazer uma cisão de página
    - Reorganização das páginas mantendo propriedades

Árvores B

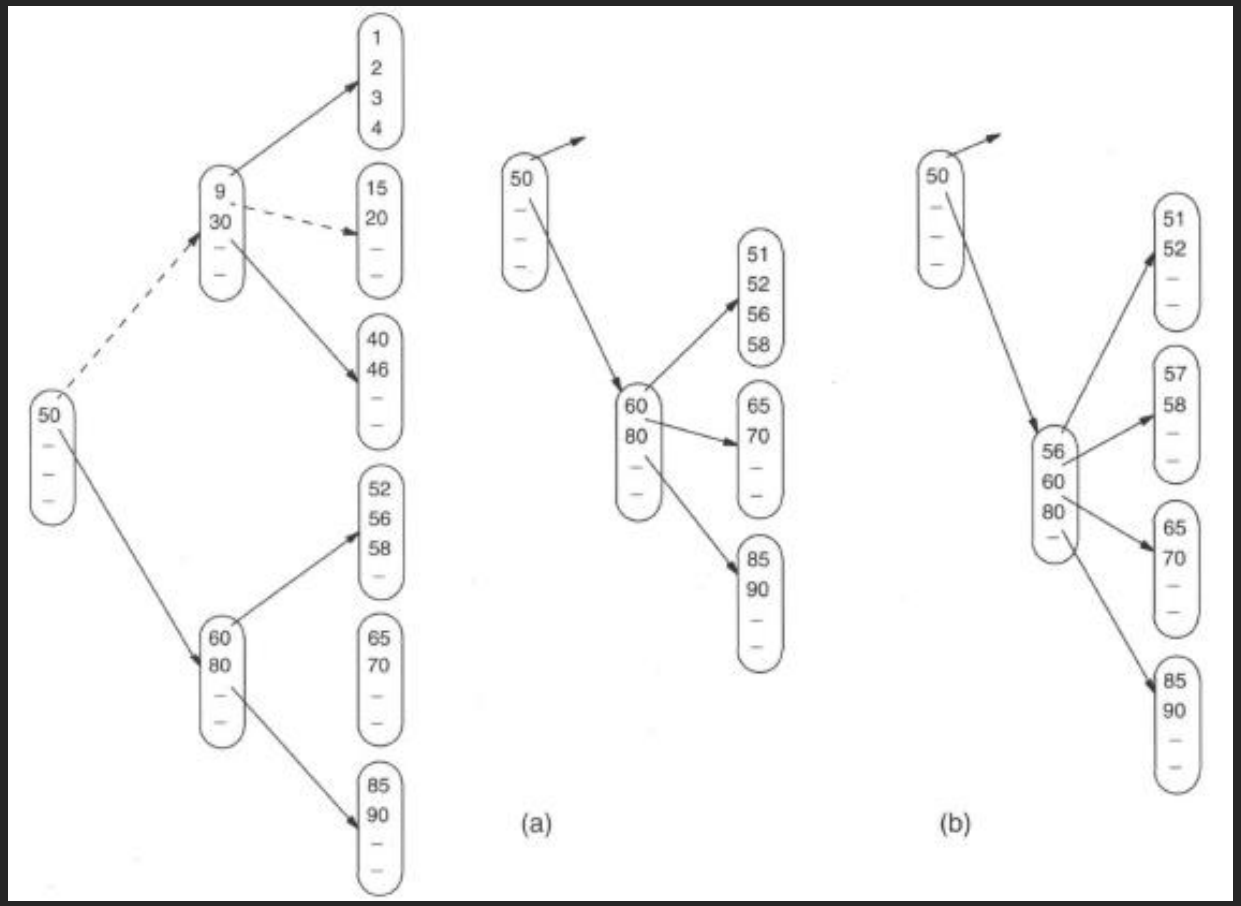
- Seja P a página a inserção resulta em 2d+1 chaves
  - Entradas de P: p_0, (s_1,p_1), (s_2,p_2), ..., (s_d,p_d), (s_{d+1},p_{d+1}),..., (s_{2d+1},p_{2d+1})
  - Cisão de página:
    - Página P fica com d chaves
      - Entradas de P: p_0, (s_1,p_1), (s_2,p_2), ..., (s_d,p_d)
    - Nova página Q é criada com d chaves
      - Entradas de Q: p_{d+1}, (s_{d+2},p_{d+2}), ..., (s_{2d+1},p_{2d+1})
    - Entrada $(s_{d+1}$,pt1) é adicionada na página W (pai de P)
      - pt1 é ponteiro para página Q: W se torna pai de Q
  - W pode necessitar de uma cisão
    - Cisões podem ser propagáveis até a raiz

Árvores B

- Algoritmo de inserção:
  - Passo 1: aplicar procedimento BuscaB, verificando validade da inserção
  - Passo 2: se a inserção é válida, incluir a nova entrada na g-ésima posição da folha P apontada por pt
  - Passo 3: verificar se a página P necessita de cisão. Em caso afirmativo, propagar a cisão enquanto necessário
- Complexidade da inserção: O(lg n)

Árvores B

- Inserção das chaves 51 (a) e 57 (b):



Árvores B

- Remoção da chave x: dois casos a considerar
  1. x está em uma folha: x é removida da página
  2. x não está em uma folha: x é substituída pela chave y imediatamente maior (necessariamente folha)
  - Em ambos os casos: remoção ocorre em folha
- Problema: após remoção, folha pode ter menos que d chaves
  - Solução: concatenação e redistribuição

Árvores B

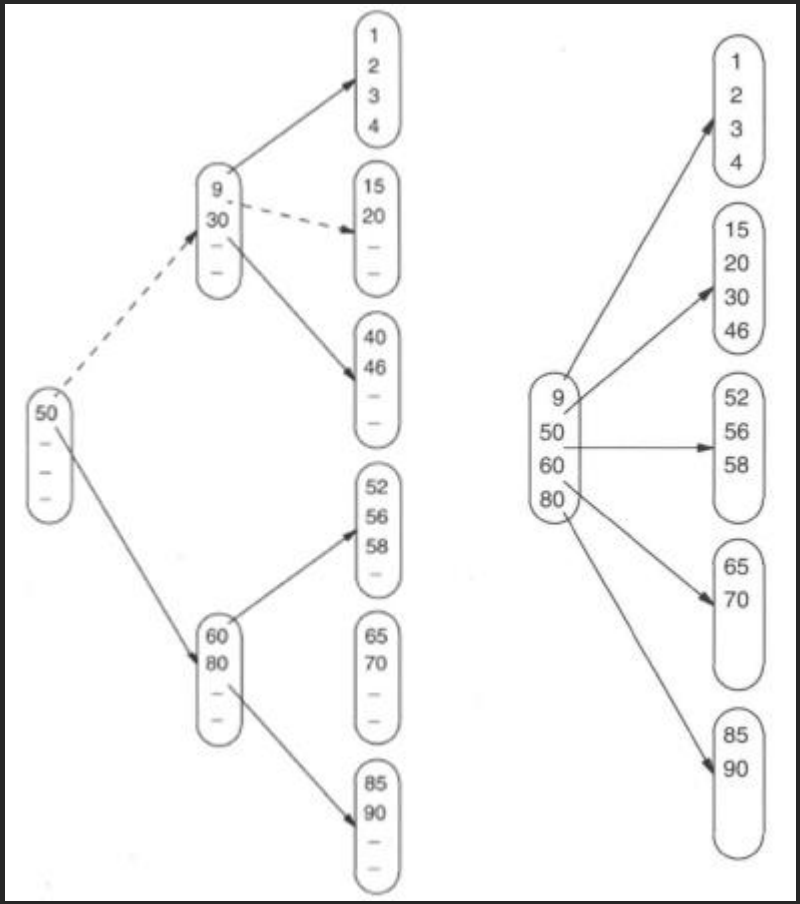
- Seja P e Q páginas irmãs adjacentes
  - P e Q tem o mesmo pai W
  - Ponteiros para P e Q são adjacentes em W
- P e Q irmãs adjacentes podem ser concatenadas se juntas têm menos de 2d chaves
  - P e Q são agrupadas em uma única página
  - W deixa de possuir uma entrada: a da chave que está entre os ponteiros P e Q
  - Chave retirada de W vai para página agrupada: no máximo 2d chaves

Árvores B

- Situação antes da concatenação:
  - W: ... (y_{j-1}, pt), $(y_j, pt1)$, (y_{j+1}, p_{j+1}), ...
    - pt é ponteiro para P e pt1 é ponteiro para Q
  - P: p_0, (s_1, p_1), ..., (s_k, p_k)
  - Q: p'_0, (s'_1, p'_1), ..., (s'_m, p'_m)
  - $k + m < 2d$
- Situação após a concatenação:
  - W: ... (y_{j-1}, pt), (y_{j+1}, p_{j+1}) ...
  - P: p_0, (s_1, p_1), ..., (s_k, p_k), (y_j, p'_0), (s'_1, p'_1), ..., (s'_m, p'_m)
- W possui uma entrada a menos: pode ter menos de 2d chaves
  - Concatenação pode se propagar até a raiz

Árvores B

- Remoção da chave 40 com propagação da concatenação:



Árvores B

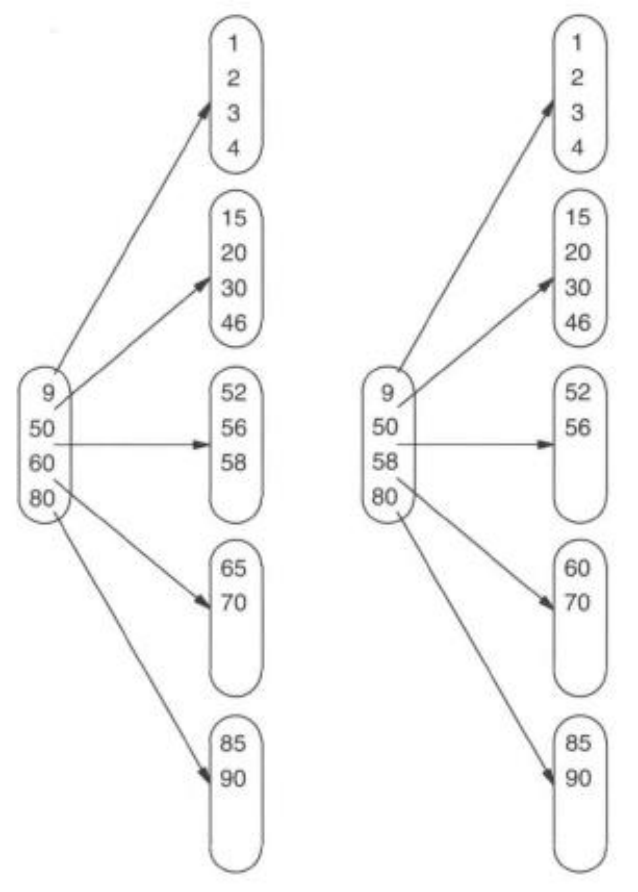
- Se irmãs adjacentes P e Q têm, juntas, mais de 2d chaves, a solução é a redistribuição de chaves
  1. P e Q são concatenadas: página P resultante muito grande
  2. É realizada uma cisão em P (considerando Q existente)
- Redistribuição não é propagável
  - W é modificada, mas mantém número de chaves

Árvores B

- Situação antes da redistribuição:
  - W: ... (y_{j-1}, pt), $(y_j, pt1)$, (y_{j+1}, p_{j+1}), ...
  - P: p_0, (s_1, p_1), ..., (s_k, p_k)
  - Q: p'_0, (s'_1, p'_1), ..., (s'_m, p'_m)
  - $k < d$ e $k+m \geq 2d$
- Situação após a redistribuição:
  - Seja $g = \lfloor (k + m)/2 \rfloor$
  - W: ... (y_{j-1}, pt), $(s'_{g-k}, pt1)$, (y_{j+1}, p_{j+1}) ...
  - P: p_0, (s_1, p_1), ..., (s_k, p_k), (y_j, p'_0), (s'_1, p'_1), ..., (s'_{g-k-1}, p'_{g-k-1})
  - Q: p'_{g-k}, (s'_{g-k+1}, p'_{g-k+1}), ..., (s'_m, p'_m)

Árvores B

- Remoção da chave 65 com redistribuição:



Árvores B

- Algoritmo de remoção da chave x:
  - Passo 1: usar buscaB para encontrar a página P que possui a chave x
  - Passo 2: se P é folha, retirar entrada que contém x. Se não é folha, buscar menor chave z que é maior que x. Seja F a página folha onde z se encontra. Substitua x por z e considere daqui em diante P = F
  - Passo 3: se P tiver menos de d entradas, execute operação de concatenação ou redistribuição
- Complexidade da remoção: O(lg n)

Exercícios

- 5.1, 5.2, 5.7, 5.9, 5.34, 5.37, 5.39, 5.40